

作业 HW4-图 实验报告

姓名：毛星博 学号：2351520 日期：2024.11.21

1. 涉及数据结构和相关背景

1.1 图的基本概念

顶点 (Vertex)：图中的基本元素，通常用 V 表示。

边 (Edge)：连接两个顶点的关系，通常用 E 表示。

有向图 (Directed Graph)：边具有方向性，从一个顶点指向另一个顶点。

无向图 (Undirected Graph)：边没有方向性，连接的两个顶点之间是双向的。

权重 (Weight)：边的一个属性，表示从一个顶点到另一个顶点的代价或距离。

邻接 (Adjacency)：如果两个顶点之间存在边，则称这两个顶点是邻接的。

度 (Degree)：与一个顶点相连的边的数量。在有向图中，分为入度 (In-degree) 和出度 (Out-degree)。

1.2 图的表示方法

邻接矩阵 (Adjacency Matrix)：使用一个二维数组表示图。

如果顶点 i 到顶点 j 有边，则矩阵对应位置为 1 (或权重值)，否则为 0。

优点：快速判断任意两个顶点是否邻接。

缺点：空间复杂度高，尤其是稀疏图。

邻接表 (Adjacency List)：每个顶点对应一个链表，链表中存储与该顶点邻接的所有顶点。

优点：节省空间，适合稀疏图。

缺点：判断两个顶点是否邻接较慢。

1.3 图的基本操作

遍历 (Traversal)：

深度优先搜索 (DFS)：类似于树的先序遍历，通过递归或栈实现。

广度优先搜索 (BFS)：类似于树的层次遍历，通过队列实现。

最短路径 (Shortest Path)：

Dijkstra 算法：适用于非负权重的有向图。

Bellman-Ford 算法：适用于含负权边的有向图。

Floyd-Warshall 算法：用于计算任意两点之间的最短路径。

最小生成树 (Minimum Spanning Tree)：

Kruskal 算法和 Prim 算法：用于无向连通图，找到权重和最小的生成树。

拓扑排序 (Topological Sorting)：

适用于有向无环图 (DAG)，用于表示任务的依赖关系。

2. 实验内容

2.1 图的遍历

2.1.1 问题描述

本题给定一个无向图，用 dfs 和 bfs 找出图的所有连通分量。

所有顶点用 0 到 $n-1$ 表示，搜索时总是从编号最小的顶点出发。使用邻接矩阵存储，或者邻接表（使用邻接表时需要使用尾插法）。

2.1.2 基本要求

输入

第 1 行输入 2 个整数 n m ，分别表示顶点数和边数，空格分割

后面 m 行，每行输入边的两个顶点编号，空格分割

输出

解释

第 1 行输出 dfs 的结果

第 2 行输出 bfs 的结果

连通子集输出格式为 $\{v_{11} v_{12} \dots\} \{v_{21} v_{22} \dots\} \dots$ 连通子集内元素之间用空格分割，子集之间无空格，'{' 和子集内第一个数字之间、'}' 和子集内最后一个元素之间、子集之间均无空格

2.1.3 数据结构设计

```
vector<vector<int>> adj(n); //邻接矩阵
set<pair<int, int>> edges; //边集合
```

2.1.4 功能说明（函数、类）

```
// 使用深度优先搜索（DFS）遍历图，获取连通分量
void dfs(int v, const vector<vector<int>>& adj, vector<bool>& visited, vector<int>& component) {
    visited[v] = true; //标记当前节点已访问
    component.push_back(v); //将当前节点加入连通分量
    for (int neighbor : adj[v]) { //遍历当前节点的邻居
        if (!visited[neighbor]) { //邻居未访问过
            dfs(neighbor, adj, visited, component); //递归调用
        }
    }
}

// 使用广度优先搜索（BFS）遍历图，获取连通分量
void bfs(int start, const vector<vector<int>>& adj, vector<bool>& visited, vector<int>& component) {
    queue<int> q; //队列
    q.push(start); //入队
```

```

        visited[start] = true;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            component.push_back(v);
            for (int neighbor : adj[v]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
    }
}

// 查找图中的所有连通分量，使用深度优先搜索（DFS）
vector<vector<int>> findConnectedComponents(const vector<vector<int>>& adj, int n) {
    vector<vector<int>> components;
    vector<bool> visited(n, false);
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            vector<int> component; // 连通分量
            dfs(i, adj, visited, component); // 深度优先搜索
            components.push_back(component); // 将连通分量加入结果集
        }
    }
    return components;
}

// 查找图中的所有连通分量，使用广度优先搜索（BFS）
vector<vector<int>> findConnectedComponentsBFS(const vector<vector<int>>& adj, int n) {
    vector<vector<int>> components;
    vector<bool> visited(n, false);
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            vector<int> component; // 连通分量
            bfs(i, adj, visited, component); // 广度优先搜索
            components.push_back(component); // 将连通分量加入结果集
        }
    }
    return components;
}

```

2.1.5 调试分析（遇到的问题 and 解决方法）

问题：未能正确识别连通分量

解决方法：检查邻接矩阵或邻接表的构建是否正确，确保所有边都被正确添加。可以通过打印邻接表或矩阵的内容来验证。

问题：DFS/BFS 未能遍历所有节点

解决方法：确保在每次调用 DFS 或 BFS 时，访问标记（visited）数组被正确更新。可以在每次访问节点时打印当前节点，以确认遍历顺序。

2.1.6 总结和体会

使用深度优先搜索（DFS）和广度优先搜索（BFS）时需要注意以下几点：

访问标记：确保每个节点在访问后被正确标记为已访问，以防止重复访问导致的无限循环。

邻接表和邻接矩阵：在使用邻接表时，注意使用尾插法插入邻接节点，确保图的结构正确。邻接矩阵需要正确初始化，确保边的权重或存在性被准确表示。

2.2 小世界现象

2.2.1 问题描述

六度空间理论又称小世界理论。理论通俗地解释为：“你和世界上任何一个陌生人之间所间隔的人不会超过 6 个人，也就是说，最多通过五个人你就能够认识任何一个陌生人。”

假如给你一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。

2.2.2 基本要求

输入

第 1 行给出两个正整数，分别表示社交网络图的结点数 N ($1 < N \leq 2000$ ，表示人数)、边数 M ($\leq 33 \times N$ ，表示社交关系数)。

随后的 M 行对应 M 条边，每行给出一对正整数，分别是该条边直接连通的两个结点的编号（节点从 1 到 N 编号）。

输出

对每个结点输出与该结点距离不超过 6 的结点数占结点总数的百分比，精确到小数点后 2 位。

每个结节点输出一行，格式为“结点编号: (空格) 百分比%”。

2.2.3 数据结构设计

```
// 数据结构设计
vector<int> adj[MAXN]; // 邻接表
bool visited[MAXN];   // 访问标记
```

2.2.4 功能说明（函数、类）

```
// 广度优先搜索,求距离不超过 6 的顶点数
```

```

int bfs(int start, int n) {
    int res = 0; // 起始点
    int distance = 0;
    queue<int> q;
    q.push(start);
    visited[start] = true;

    // 层次化遍历
    while (!q.empty() && distance <= 6) {
        int levelSize = q.size(); // 当前层的节点数量
        for (int i = 0; i < levelSize; i++) {
            int v = q.front();
            q.pop();
            res++; // 每处理一个顶点，结果计数加 1
            for (int j : adj[v]) {
                if (!visited[j]) {
                    visited[j] = true;
                    q.push(j);
                }
            }
        }
        distance++; // 处理完当前层后，再递增距离
    }
    return res;
}

```

2.2.5 调试分析（遇到的问题 and 解决方法）

问题：BFS 未能正确计算距离

解决方法：确保在 BFS 过程中，距离计数器在每层遍历后递增（我原先的错误是每出栈一个就增加计数器），并且在入队时标记节点为已访问。可以通过打印每层的节点数量来验证。

2.2.6 总结和体会

使用广度优先搜索（BFS）时需注意：

距离限制：确保在 BFS 过程中正确控制距离的层次，避免超出 6 层的搜索。

访问标记的重置：每次进行 BFS 时，需要重置访问标记，以确保每次搜索都是从新节点开始。

计算距离：距离计数器要在每层遍历后递增，这需要考虑层的节点数

2.3 村村通

2.3.1 问题描述

村村通

描述

N 个村庄，从 1 到 N 编号，现在请你修建一些路使得任何两个村庄都彼此连通。我们称两个村庄 A 和 B 是连通的，当且仅当在 A 和 B 之间存在一条路，或者存在一个村庄 C，使得 A 和 C 之间有一条路，并且 C 和 B 是连通的。

已知在一些村庄之间已经有了一些路，您的工作是再兴建一些路，使得所有的村庄都是连通的，并且新建的路的长度是最小的。

2.3.2 基本要求

输入

第一行包含一个整数 n ($3 \leq n \leq 100$)，表示村庄数目。

接下来 n 行,每行 n 个非负整数，表示村庄 i 和村庄 j 之间的距离。距离值在[1,1000]之间。

接着是一个整数 m，后面给出 m 行，每行包含两个整数 a,b,($1 \leq a < b$),表示在村庄 a 和 b 之间已经修建了路。

输出

输出一行，仅有一个整数，表示为使所有的村庄连通，要新建公路的长度的最小值。

2.3.3 数据结构设计

```
// 数据结构设计

struct Edge {          // 边
    int u, v, weight; // u -> v, weight w
    Edge(int u, int v, int w) : u(u), v(v), weight(w) {}
};

vector<vector<int>>> G; // 邻接矩阵
vector<Edge> E;        // 边集
```

2.3.4 功能说明（函数、类）

```
// 并查集

class UnionFind {
private:
    vector<int> parent; // 父节点
public:
    UnionFind(int n) : parent(n) { // 初始化，每个节点的父节点为自身
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) { // 路径压缩
        if (parent[x] == x) {
            return x;
        }
    }
}
```

```

        return find(parent[x]); // 递归查找父节点
    }

    void unionSets(int x, int y) { // 合并两个集合
        int px = find(x);
        int py = find(y);
        if (px != py) {
            parent[px] = py; // 合并两个集合
        }
    }
};

// Kruskal 算法求最小生成树
int kruskal(int n, vector<Edge>& edges) {
    UnionFind uf(n);

    sort(edges.begin(), edges.end(), [](Edge a, Edge b) { return a.weight < b.weight; }); // 按权重排序
    int totalWeight = 0;
    for (const Edge& edge : edges) { // 遍历所有边
        if (uf.find(edge.u) != uf.find(edge.v)) { // 若不属于同一集合
            uf.unionSets(edge.u, edge.v); // 合并两个集合
            totalWeight += edge.weight; // 加权重
        }
    }
    return totalWeight;
}

```

2.3.5 调试分析（遇到的问题和解决方法）

问题：原先使用 Kruskal 算法中用到深度优先遍历来查找两个节点是否连通，但是提交作业时发现问题程序运行超时，每次都深搜效率太低

解决方法：查询资料后，改为维护一个并查集查找两个节点是否连通，提高程序效率。

2.3.6 总结和体会

在实现最小生成树时，使用 Kruskal 算法时需注意：

边的排序：确保在合并集合之前，边按照权重正确排序，以保证选择最小权重的边。

并查集的实现：并查集的路径压缩和合并操作需要正确实现，以提高效率。

图的连通性：在添加新边时，需检查是否会形成环，确保最终生成的图是连通的。

2.4 给定条件下构造矩阵

2.4.1 问题描述

给你一个正整数 k ，同时给你：

一个大小为 n 的二维整数数组 `rowConditions`，其中 `rowConditions[i] = [abovei, belowi]` 和

一个大小为 m 的二维整数数组 `colConditions`，其中 `colConditions[i] = [lefti, righti]`。两个数组里的整数都是 1 到 k 之间的数字。

你需要构造一个 $k \times k$ 的矩阵，1 到 k 每个数字需要恰好出现一次。剩余的数字都是 0。

矩阵还需要满足以下条件：

对于所有 0 到 $n - 1$ 之间的下标 i ，数字 `abovei` 所在的行必须在数字 `belowi` 所在行的上面。

对于所有 0 到 $m - 1$ 之间的下标 i ，数字 `lefti` 所在的列必须在数字 `righti` 所在列的左边。

返回满足上述要求的矩阵，题目保证若矩阵存在则一定唯一；如果不存在答案，返回一个空矩阵。

2.4.2 基本要求

输入

第一行包含 3 个整数 k 、 n 和 m

接下来 n 行，每行两个整数 `abovei`、`belowi`，描述 `rowConditions` 数组

接下来 m 行，每行两个整数 `lefti`、`righti`，描述 `colConditions` 数组

输出

如果可以构造矩阵，打印矩阵；否则输出 -1

矩阵中每行元素使用空格分隔

2.4.3 数据结构设计

```
vector<pair<int, int>> rowConditions(n);           // 行条件
vector<pair<int, int>> colConditions(m);           // 列条件
vector<vector<int>> matrix(k, vector<int>(k, 0)); // 矩阵
vector<int> rowlist(k);                           // 行拓扑排序
vector<int> collist(k);                           // 列拓扑排序
```

2.4.4 功能说明（函数、类）

```
// 拓扑排序
bool topo(int k, int num, vector<pair<int, int>>& Conditions, vector<int>& list) {
    vector<int> indegree(k, 0); // 储存每个顶点的入度
    vector<vector<int>> edg(k); // 储存每个顶点的出边
    for (auto& p : Conditions) {
        indegree[p.second]++; // 入度+1
        edg[p.first].push_back(p.second);
    }
    queue<int> q;
    for (int i = 0; i < k; i++) {
        if (indegree[i] == 0) q.push(i); // 入度为 0 的顶点入队列
    }
    int cnt = 0;
    while (!q.empty()) {
        int u = q.front(); // 取出队首顶点
        q.pop();
```

```

        list[cnt++] = u;    // 加入拓扑排序序列

        for (int v : edg[u]) {    // 遍历 u 的出边
            indegree[v]--;

            if (indegree[v] == 0) q.push(v); // 入度为 0 的顶点入队列
        }
    }

    if (cnt != k) return false; // 有环，无法拓扑排序
    return true;
}

// 构造矩阵
void buildMatrix(int k, vector<vector<int>>& matrix, vector<int>& rowlist, vector<int>& collist) {
    for (int i = 0; i < k; i++) {
        int row, col;

        for (int j = 0; j < k; j++) {
            if (i == rowlist[j]) { // 找到行 i 在矩阵中的位置
                row = j;
                break;
            }
        }

        for (int j = 0; j < k; j++) {
            if (i == collist[j]) { // 找到列 i 在矩阵中的位置
                col = j;
                break;
            }
        }

        matrix[row][col] = i + 1;
    }
}

```

2.4.5 调试分析（遇到的问题解决方法）

问题：无法检查是否能够成功构建矩阵

解决方法：检查图中是否存在环（如果加入拓扑排序的节点与总数不相同，说明有环，矩阵构建不了）；确保入度和出度的计算正确。可以在拓扑排序前后打印入度数组以进行验证

2.4.6 总结和体会

拓扑排序的实现需注意：

入度和出度的计算：确保正确计算每个节点的入度，以便进行拓扑排序。

环的检测：在拓扑排序过程中，若无法完成排序，则说明存在环，需返回错误。

矩阵的构建：在构建矩阵时，确保行列条件得到满足，避免位置错误。

另外，本题限定了矩阵如果能够构建则是唯一的，如果没有这个限制，拓扑排序中可能不会包含所有数，需要自行填充没有限定的数。

2.5 必修课

2.5.1 问题描述

某校的计算机系有 n 门必修课程。学生需要修完所有必修课程才能毕业。

每门课程都需要一定的学时去完成。有些课程有前置课程，需要先修完它们才能修这些课程；而其他课程没有。不同于大多数学校，学生可以在任何时候进行选课，且同时选课的数量没有限制。

现在校方想要知道：

从入学开始，每门课程最早可能完成的时间（单位：学时）；

对每一门课程，若将该课程的学时增加 1，是否会延长入学到毕业的最短时间。

2.5.2 基本要求

输入

第一行，一个正整数 n ，代表课程的数量。

接下来 n 行，每行若干个整数：

第一个整数为 t_i ，表示修完该课程所需的学时。

第二个整数为 c_i ，表示该课程的前置课程数量。

接下来 c_i 个互不相同的整数，表示该课程的前置课程的编号。

该校保证，每名入学的学生，一定能够在有限的时间内毕业。

输出

输出共 n 行，第 i 行包含两个整数：

第一个整数表示编号为 i 的课程最早可能完成的时间。

第二个整数表示，如果将该课程的学时增加 1，入学到毕业的最短时间是否会增加。如果会增加则输出 11，否则输出 00。

每行的两个整数以一个空格隔开。

2.5.3 数据结构设计

```
vector<vector<int>> Condition(n);    // 课程的先修课程
vector<vector<int>> reCondition(n);  // 课程的后修课程
vector<int> ve(n, 0);                // 最早开始时间
vector<int> vl(n, 0);                // 最晚开始时间
vector<int> time(n);                 // 课程持续时间
```

2.5.4 功能说明（函数、类）

```
class Solution {
public:
    // 求最早开始时间
    int Ve(int n, vector<vector<int>>& reCondition, vector<int>& ve, vector<int>& time) {
        for (int i = 0; i < n; i++) {
            ve[i] = 0; // 最早开始时间初始化为 0
        }
        vector<int> indegree(n, 0); // 入度初始化为 0
```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < reCondition[i].size(); j++) {
            indegree[reCondition[i][j]]++; // 计算每个课程的入度
        }
    }

    queue<int> q; // 队列
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) { // 如果入度为 0, 则将其加入队列
            q.push(i);
        }
    }

    while (!q.empty()) { // 队列不为空
        int u = q.front();
        q.pop();

        for (int i = 0; i < reCondition[u].size();
            i++) { // 遍历 u 的后修课程
            int v = reCondition[u][i];
            indegree[v]--; // 入度减 1
            if (indegree[v] == 0) { // 如果入度为 0, 则将其加入队列
                q.push(v);
            }
            if (ve[v] < ve[u] + time[u]) {
                // 如果 v 的最早开始时间小于 u 的最早开始时间加上 u 的持续时间, 则更新 v 的最早开始时间
                ve[v] = ve[u] + time[u];
            }
        }
    }

    return *max_element(ve.begin(), ve.end()); // 返回最后一个课程的最早开始时间
}

// 求最晚开始时间
void V1(int n, vector<vector<int>>& Condition, vector<int>& v1, vector<int>& time, int maxVe) {
    vector<int> outdegree(n, 0); // 出度初始化为 0
    v1.assign(n, maxVe); // 最晚开始时间初始化为最大的最早开始时间
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < Condition[i].size(); j++) {
            outdegree[Condition[i][j]]++; // 计算每个课程的出度
        }
    }

    stack<int> s; // 栈
    for (int i = 0; i < n; i++) {
        if (outdegree[i] == 0) { // 如果出度为 0, 则将其加入栈
            s.push(i);
        }
    }
}

```

```

    }
    while (!s.empty()) { // 栈不为空
        int u = s.top();
        s.pop();
        for (int i = 0; i < Condition[u].size(); i++) { // 遍历 u 的先修课程
            int v = Condition[u][i];
            outdegree[v]--; // 出度减 1
            if (outdegree[v] == 0) { // 如果出度为 0，则将其加入栈
                s.push(v);
            }
            if (vl[v] > vl[u] - time[v]) {
                // 如果 v 的最晚开始时间大于 u 的最晚开始时间减去 u 的持续时间，则更新 v 的最晚开始时间
                vl[v] = vl[u] - time[v];
            }
        }
    }
}
};

```

2.5.5 调试分析（遇到的问题解决方法）

问题：最晚开始时间计算错误

解决方法：发现判断条件 $vl[v] > vl[u] - time[v]$ 错误的写成了 $vl[v] > vl[u] - time[u]$ 。由于 v 是 u 的先修课程，时间依赖 v ，所以要考虑的是 v 的时间

2.5.6 总结和体会

实现关键路径算法时，要注意事件最早开始时间和最晚开始时间的不同判断条件。最早开始时间是前一个最早开始时间加上活动时间的最大值，最晚开始时间是后一个最晚开始时间减去活动时间的最小值。另外，本题实际上简化了关键路径，因为所有活动的权重只与发出他的事件有关，因此只需求事件的最早最晚时间，不用再求活动的。

2.6 小马吃草

2.6.1 问题描述

假设无向图 G 上有 N 个点和 M 条边，点编号为 1 到 N ，第 i 条边长度为 w_i ，其中 H 个点上有可以食用的牧草。另外有 R 匹小马，第 j 匹小马位于点 $start_j$ ，需要前往任意一个有牧草的点进食牧草，然后前往点 end_j ，请你计算每一匹小马需要走过的最短距离。

2.6.2 基本要求

输入描述

第一行两个整数 N 、 M ，分别表示点和边的数量

接下来 M 行，第 i 行包含三个整数 x_i, y_i, w_i ，表示从点 x_i 到点 y_i 有一条长度为 w_i 的边，保

证 $x_i \neq y_i$

接下来一行有两个整数 H 和 R ，分别表示有牧草的点数量和小马的数量

接下来一行包含 H 个整数，为 H 个有牧草的点编号

接下来 R 行，第 j 行包含两个整数 $start_j$ 和 end_j ，表示第 j 匹小马起始位置和终点位置

题目保证两个点之间一定是连通的，并且至少有一个点上有牧草

输出描述

输出共 R 行，表示 R 匹小马需要走过的最短距离

2.6.3 数据结构设计

```
#define MAXN 1005 // 最大顶点数
const int INF = numeric_limits<int>::max();

// 数据结构定义
struct Edge {
    int dest; // 目的顶点
    int weight; // 边的权值
    Edge(int _dest, int _weight) : dest(_dest), weight(_weight) {} // 构造函数
};

vector<Edge> G[MAXN]; // 邻接表
int dist[MAXN][MAXN]; // dist[i][j] 表示从 i 到 j 的最短路径
```

2.6.4 功能说明（函数、类）

```
// Dijkstra 算法求最短路径
void dijkstra(int start) {
    fill(dist[start], dist[start] + MAXN, INF); // 初始化 dist 数组
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq; // 优先队列
    pq.push({0, start}); // 初始源点到源点的距离为 0
    dist[start][start] = 0; // 初始源点到源点的距离为 0
    while (!pq.empty()) {
        int u = pq.top().second; // 取出当前最小距离的顶点
        int d = pq.top().first; // 取出当前最小距离
        pq.pop(); // 弹出当前顶点
        if (d > dist[start][u]) continue; // 如果距离大于当前最短距离，则跳过
        for (int i = 0; i < G[u].size(); i++) {
            int v = G[u][i].dest; // 遍历 u 的邻居
            int w = G[u][i].weight; // 遍历 u 的邻居的权值
            if (dist[start][u] + w < dist[start][v]) { // 如果经过 u 的距离更短，则更新距离
                dist[start][v] = dist[start][u] + w; // 更新距离
                pq.push({dist[start][v], v}); // 加入优先队列
            }
        }
    }
}
```

2.6.5 调试分析（遇到的问题和解决方法）

问题：Dijkstra 算法未能正确计算最短路径

解决方法：原先未能每次取出队列中最小的值，查询资料后改为使用优先队列来获取队列最小值。

2.6.6 总结和体会

本题使用的算法是 Dijkstra 算法，遍历所有点来获取其到每一点的最短距离，这种算法本质上是贪心算法，通过局部最优解一步步得到全局最优。其实本题也可以使用 Floyd 算法，直接求所有点之间的距离最小值，可能效率会更高。

3. 实验总结

在本次图相关的实验中，我深入探讨了图的基本概念、表示方法及其常用算法，涵盖了从图的遍历到最小生成树、拓扑排序等多个方面。

通过对图的基本概念的学习，我掌握了顶点、边、有向图与无向图、权重等基本术语，并理解了邻接矩阵和邻接表的优缺点。这为后续的图算法实现奠定了基础。

在图的遍历实验中，我实现了深度优先搜索（DFS）和广度优先搜索（BFS）来找出图的所有连通分量。通过调试，我发现了在标记访问状态和遍历邻接节点时可能出现的问题，并通过打印调试信息来解决这些问题。这一过程不仅加深了我对图遍历算法的理解，也提高了我的调试能力。

在小世界现象的实验中，我计算了社交网络中每个节点与其他节点的距离，并分析了符合“六度空间”理论的节点占总节点的百分比。我学习了如何使用广度优先搜索来实现这一功能，并在调试过程中发现了距离计算和输出格式的问题，最终成功输出了符合要求的结果。

通过村村通的实验，我应用了 Kruskal 算法来构建最小生成树，确保所有村庄之间的连通性。我在实现并查集时，遇到了合并集合的逻辑问题，通过细致的调试和验证，确保了算法的正确性。这一实验让我体会到了图算法在实际问题中的应用。

在构造矩阵的实验中，我实现了拓扑排序来满足行列条件。通过对条件的分析和实现，我发现了拓扑排序中的环检测问题，并通过调整入度和出度的计算来解决。这个实验增强了我对拓扑排序的理解及其在约束条件下的应用。

在必修课的实验中，我通过计算课程的最早和最晚开始时间，分析了课程之间的依赖关系。我实现了一个基于拓扑排序的算法，并在调试过程中发现了入度和出度计算的错误，最终成功输出了每门课程的最早完成时间及其对毕业时间的影响。

在小马吃草的实验中，我使用 Dijkstra 算法计算每匹小马的最短路径。通过对优先队列的使用和距离数组的更新，我们解决了路径计算中的问题，并成功输出了每匹小马的最短距离。这一实验让我进一步理解了最短路径算法的实际应用。

通过本次实验，我不仅掌握了图的基本概念和算法实现，还提高了调试和解决问题的能力。每个实验都让我体验到了理论与实践的结合，增强了对图论的理解和应用能力。未来，我将继续深入学习图算法及其在更复杂问题中的应用，以提升我的编程和算法设计能力。